BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 1

Roll no:43 PRN No:12410889 Name of Student: Akshada Ganesh Sadaphal

Subject Name: Deep Learning

Problem Statement: To install and configure TensorFlow and Keras in Google Colab and perform the essential data preprocessing steps on the Fashion MNIST dataset. The experiment includes loading the dataset, exploring its structure, normalizing pixel values, splitting the data into training and testing sets (already provided in the dataset), and visualizing sample images to prepare the data for deep learning model development.

Algorithm:

- Open Google Colab.
- Import TensorFlow, Keras, NumPy, and Matplotlib libraries.
- Verify the TensorFlow installation and version.
- Load the Fashion MNIST dataset using Keras.
- Display the shape of the training and testing datasets.
- Visualize sample images from the dataset.
- Normalize the pixel values by dividing them by 255.
- Verify the normalized pixel value range.
- Display the class labels corresponding to the images.
- The dataset is now preprocessed and ready for model training.

Code:

Step 1: Import Libraries

```
import tensorflow as tf

import matplotlib.pyplot as plt

import numpy as np
```

Step 2: Check TensorFlow Version

```
print("TensorFlow Version:", tf.__version__)
```

Step 3: Load Fashion MNIST Dataset

```
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.fashion_mnist.load_data()
```

Step 4: Display Dataset Information

```
print("Training Images Shape :", train_images.shape)

print("Training Labels Shape :", train_labels.shape)

print("Testing Images Shape :", test_images.shape)

print("Testing Labels Shape :", test_labels.shape)
```

Step 5: Display First 10 Labels

```
print("\nFirst 10 Labels:")

print(train_labels[:10])
```

Step 6: Normalize the Dataset Scale pixel values from 0-255 to 0-1

```
train_images = train_images / 255.0

test_images = test_images / 255.0
```

Step 7: Verify Normalization

```
print("\nMinimum Pixel Value :", train_images.min())

print("Maximum Pixel Value :", train_images.max())
```

Step 8: Dataset Information After Normalization

```
print("\nTraining Images Shape :", train_images.shape)
print("Testing Images Shape :", test_images.shape)
print("Training Labels :", train_labels)

class_names = [  'T-shirt/Top', 'Trouser', 'Pullover',  'Dress', 'Coat', 'Sandal',   'Shirt',
   'Sneaker','Bag',    'Ankle Boot']
```

Step 9: Display One Fashion Image

```
plt.imshow(train_images[2], cmap='gray')
plt.title("Sample Fashion Image : " + class_names[train_labels[2]])
plt.axis('off')
plt.show()
```

Step 10: Display First 10 Images

```
plt.figure(figsize=(10,6))
for i in range(10):
   plt.subplot(2,5,i+1)
   plt.imshow(train_images[i], cmap='gray')
   plt.title(class_names[train_labels[i]])
   plt.axis('off')

plt.tight_layout()
plt.show()
```

Step 11: Create Neural Network Model

```
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28,28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))
```

Step 12: Compile the Model

```
my_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Step 13: Train the Model

```
history = my_model.fit(
    train_images,
    train_labels,
    epochs=3
)
```

Step 14: Evaluate the Model

```
test_loss, test_accuracy = my_model.evaluate(test_images, test_labels)
print("\nTest Accuracy :", test_accuracy)
print("Test Loss :", test_loss)
```

Step 15: Predict on Test Data

```
prediction = my_model.predict(test_images)
print("\nPredicted Class :", class_names[np.argmax(prediction[0])])
print("Actual Class :", class_names[test_labels[0]])
```

Step 16: Display Predicted Image

```
plt.imshow(test_images[0], cmap='gray')
plt.title("Predicted : " + class_names[np.argmax(prediction[0])])
plt.axis('off')
plt.show()
```

Step 17: Save the Model

```
my_model.save("fashion_mnist_model.keras")
print("\nModel saved successfully as 'fashion_mnist_model.keras'")
```

Output:

```
[1] ✓ 4s
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
```

```
[7] ✓ 1s
# Load Fashion MNIST Dataset

(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
[8] ✓ 0s
print("Training Images Shape :", train_images.shape)
print("Training Labels Shape :", train_labels.shape)

print("Testing Images Shape :", test_images.shape)
print("Testing Labels Shape :", test_labels.shape)
```

```
Training Images Shape : (60000, 28, 28)
Training Labels Shape : (60000,)
Testing Images Shape : (10000, 28, 28)
Testing Labels Shape : (10000,)
```

```
[9] ✓ 0s
# Scale pixel values from 0-255 to 0-1

train_images = train_images / 255.0
test_images = test_images / 255.0
```

```
[10] ✓ 0s
print("Minimum Pixel Value :", train_images.min())
print("Maximum Pixel Value :", train_images.max())
```

```
Minimum Pixel Value : 0.0
Maximum Pixel Value : 1.0
```

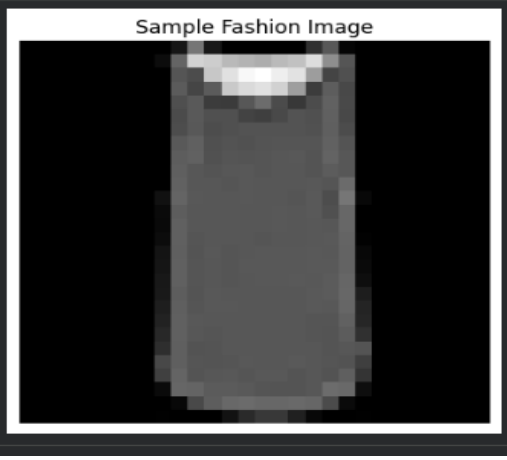

```
[10] ✓ 0s
print("Minimum Pixel Value :", train_images.min())
print("Maximum Pixel Value :", train_images.max())
```

```
Minimum Pixel Value : 0.0
Maximum Pixel Value : 1.0
```


```
[11] ✓ 0s
print(train_images.shape)
print(test_images.shape)
print(train_labels)
```

```
(60000, 28, 28)
(10000, 28, 28)
[9 0 0 ... 3 0 5]
```

```
[12] ✓ 0s
plt.imshow(train_images[2], cmap='gray')
plt.title("Sample Fashion Image")
plt.axis('off')
plt.show()
```

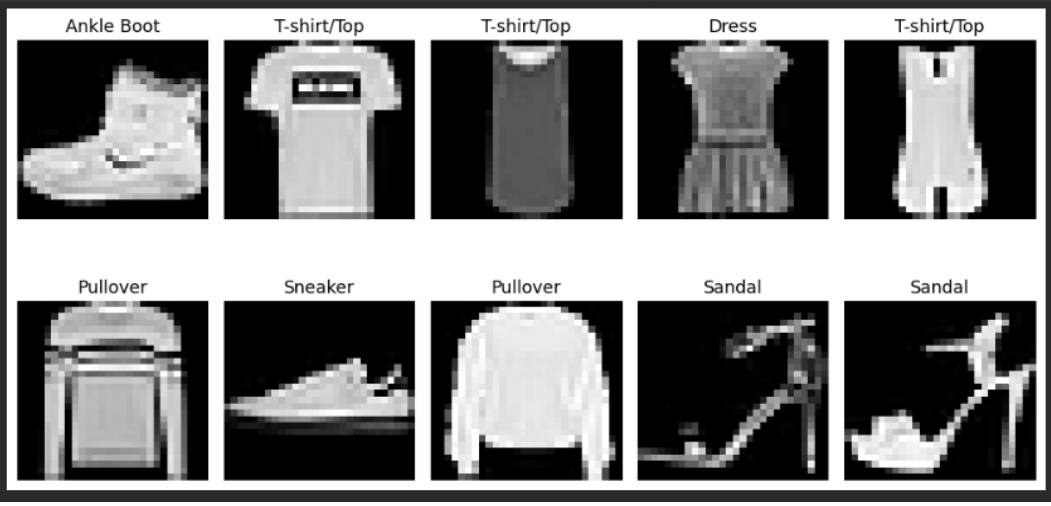


[13]

```
class_names = ['T-shirt/Top', 'Trouser','Pullover', 'Dress','Coat','Sandal', 'Shirt',  'Sneaker', 'Bag', 'Ankle Boot']

plt.figure(figsize=(10,6))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(train_images[i], cmap='gray')
    plt.title(class_names[train_labels[i]])
    plt.axis('off')

plt.tight_layout()
plt.show()
```



[14]

```
my_model = tf.keras.models.Sequential()

my_model.add(tf.keras.layers.Flatten(input_shape=(28,28)))

my_model.add(tf.keras.layers.Dense(128, activation='relu'))

my_model.add(tf.keras.layers.Dense(10, activation='softmax'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the
  super().__init__(**kwargs)
```

[15]

```
my_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

[16]

```
history = my_model.fit(
    train_images,
    train_labels,
    epochs=3
)
```

```
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 5ms/step - accuracy: 0.8235 - loss: 0.4990
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8640 - loss: 0.3752
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.8766 - loss: 0.3367
```

[17]

```
test_loss, test_accuracy = my_model.evaluate(test_images, test_labels)

print("Test Accuracy :", test_accuracy)
print("Test Loss :", test_loss)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8692 - loss: 0.3636
Test Accuracy : 0.8691999912261963
Test Loss : 0.3636046648025127
```

[18]
✓ 0s

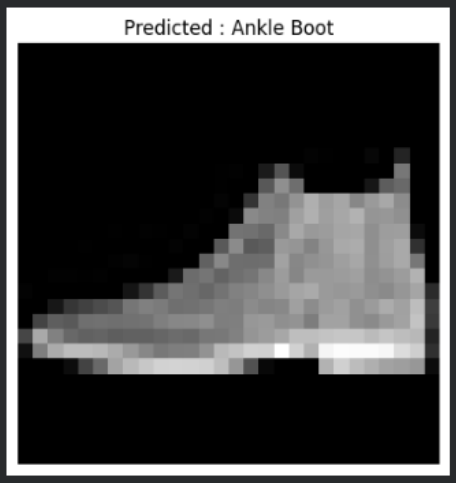
```
prediction = my_model.predict(test_images)

print("Predicted Class :", np.argmax(prediction[0]))
print("Actual Class :", test_labels[0])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
Predicted Class : 9
Actual Class : 9
```

[19]
✓ 0s

```
plt.imshow(test_images[0], cmap='gray')
plt.title("Predicted : " + class_names[np.argmax(prediction[0])])
plt.axis('off')
plt.show()
```



[20]
✓ 0s

```
my_model.save("fashion_mnist_model.keras")
```